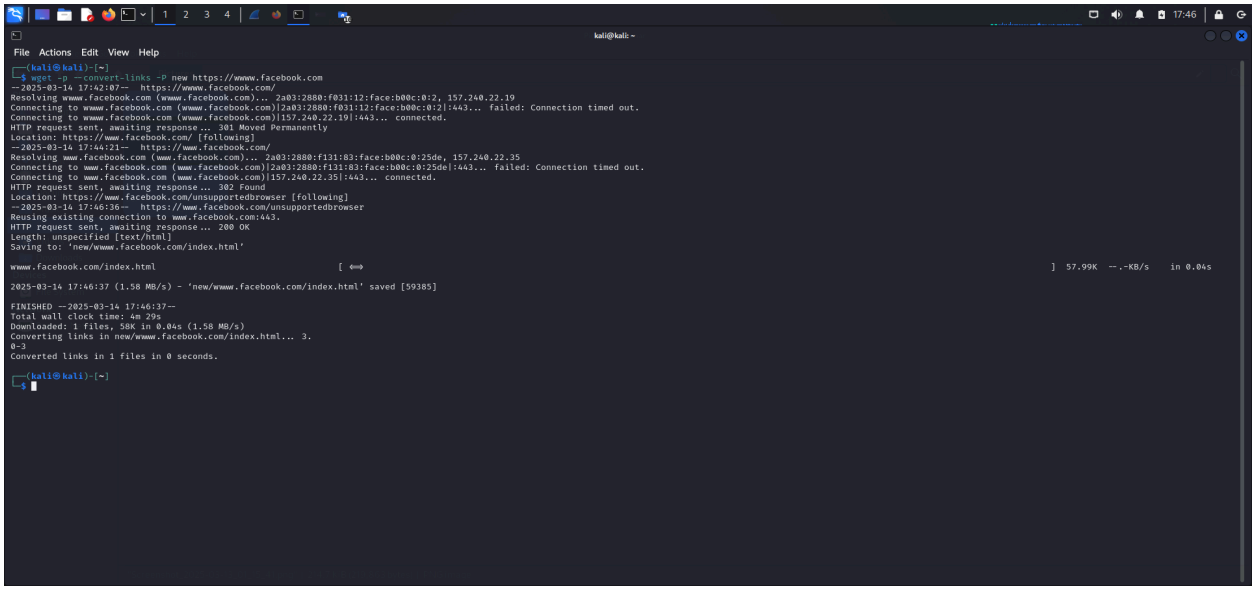
Abdul Gafoor Mubashir
Assignment 4

Abstract
This project demonstrates the use of firewall rules to control inbound and outbound network traffic by blocking, redirecting, and filtering packets based on IP and MAC addresses. Testing revealed that MAC address filtering is ineffective against attackers due to MAC spoofing. Additionally, firewall rules successfully blocked SSH and HTTP traffic. The results highlight both the strengths and limitations of IPTables in enforcing network security policies.

Introduction
Using Virtualbox to run two different operating systems as Virtual machines. Using Kali Linux virtual machine and an Ubuntu virtual machine for this project. Kali will be the attacking system. Ubuntu will be the target system. Networks are being set on Bridged adapter mode. Using IPTables, a command-line utility in Linux used for configuring and managing firewall rules. And Macchanger to change or spoof mac addresses. SSH "Secure Shell" is used for communication between these two systems. And IPTables to port blocking and redirecting.

-All commands used are written in a separate file attached to the project.

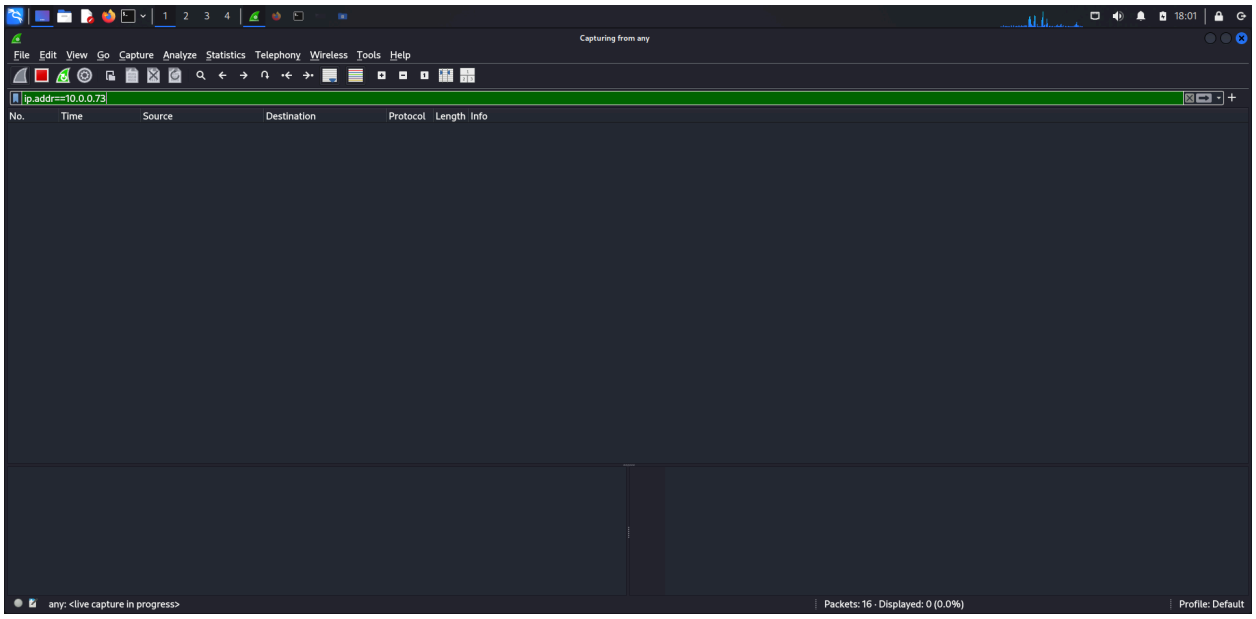
Summary of Results:
Now run both Kali Linux and Ubuntu Vm's and before starting both Vm's, change the network to Bridged Adapter mode and connect both Vm's to the Network. And also turn on Promiscuous mode. Now open Kali Linux and open Bash then type "wget -p –convert -links -P new https://www.facebook.com" this will create a folder called 'new' and attempt to download a webpage using wget function which saves all necessary files (CSS, images, JavaScript) to properly display the HTML page in the new folder.



The above image displays that using the wget function, we are downloading all the required html files of Facebook.com in the new directory.

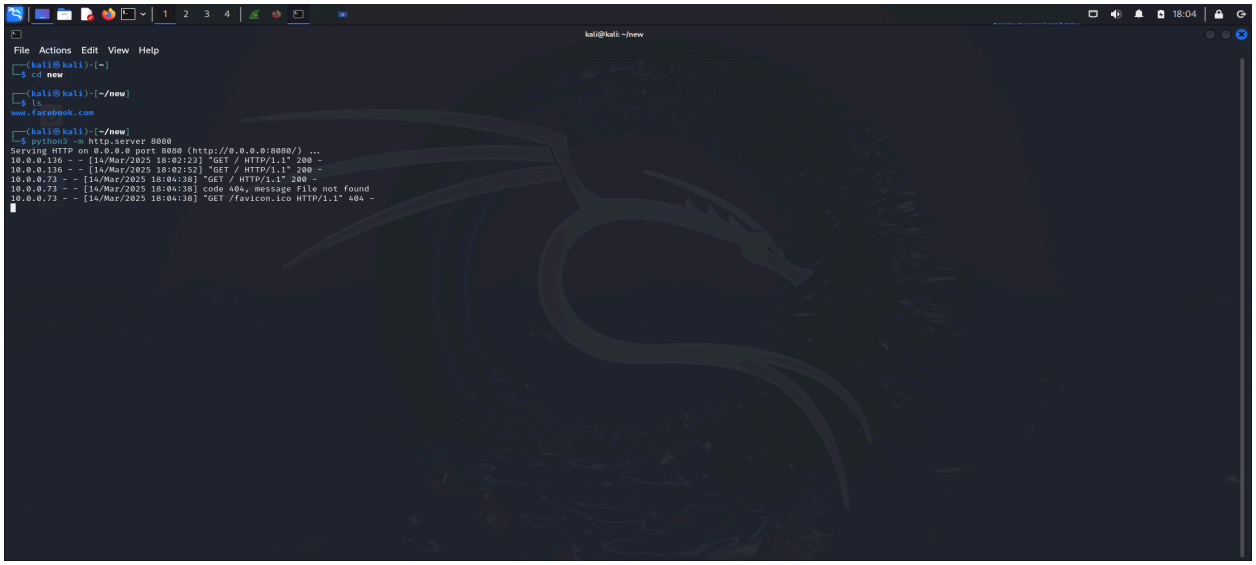
Now after we download the required files to host a website we are going to check our ip address using ip addr as shown in the below image. And our Kali Terminal ip address is “10.0.0.99”.

After that we are going to open wireshark-a network monitoring tool in Kali Linux, and change its setting to capture ‘any’ file. Once wireshark starts we are going to change the filter setting to only capture the packets from ubuntu terminal ip address which is 10.0.0.73.



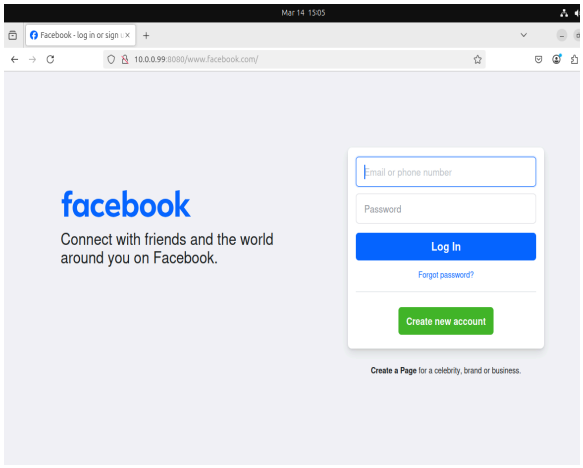
The above image shows the wireshark is set to filter 10.0.0.73 ip data capture.

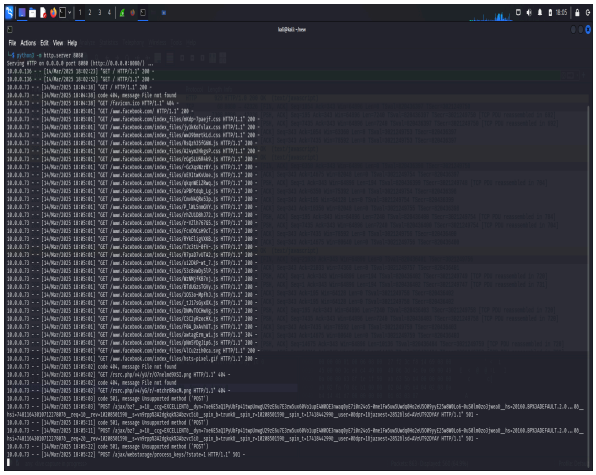
Once wireshark is set up. We are going to open another bash terminal in Kali and type the command “cd new”. This command is used in the terminal to change the current working directory to a folder named new. Then we are going to check the directory using “ls”;This will list files and folders in your current directory. After that, we are going to host the website using a python command called “python3 -m http.server 8080”. This command will start hosting the website using the files in the current directory over HTTP on port 8080.



The above image shows the cd and ls command being used and also hosting the website using the python command.

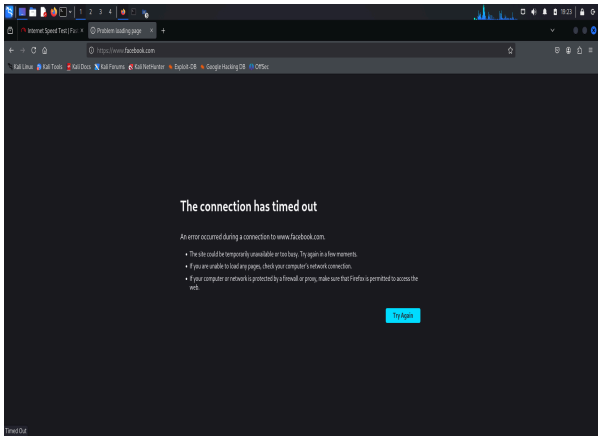
Once we host our website on Kali using python command, Now try to open Ubuntu terminal and go to browser, and type the command http://10.0.0.99:8080, this will open the hosted website by kali and show us exact website of facebook which is fake, and when we click anything here, everything will be logged in the kali terminal.





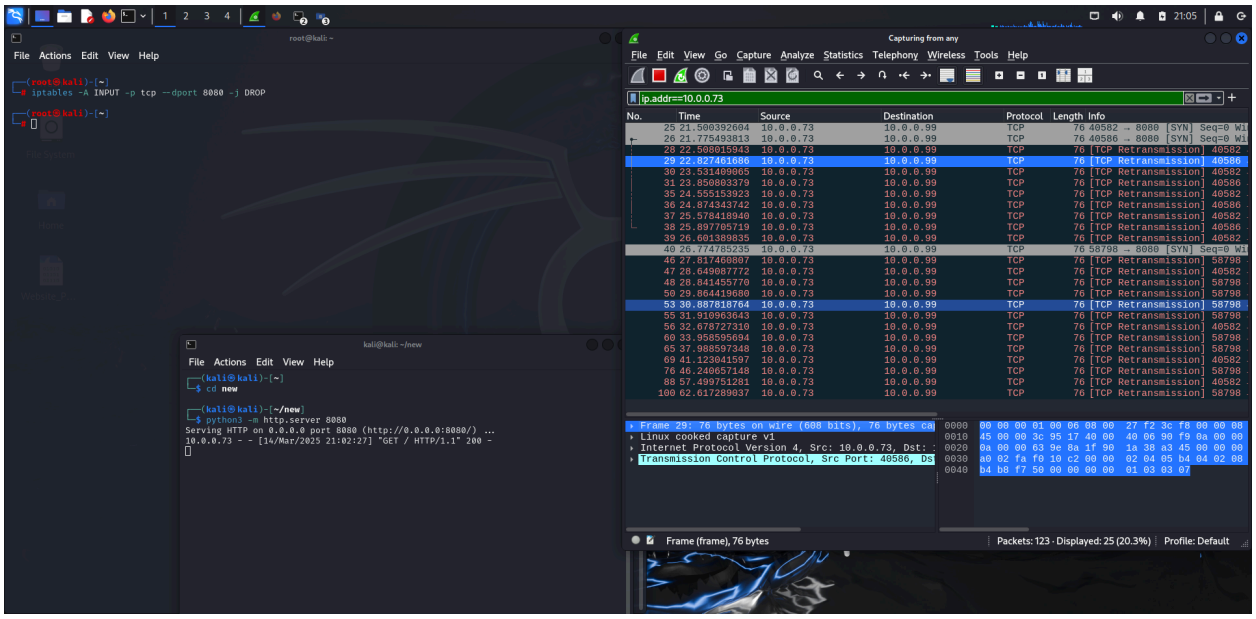
The first image shows a fake facebook website hosted from the kali python server which is being accessed from the Ubuntu web browser. The second one shows all the logs being filed in the Kali hosted terminal as we access the website.

After we've learnt how to host a website from kali, now we are going to block all the outbound traffic for the real facebook server from our network. To do that we are going to know the host ipv4 address using the command "host -t a www.facebook.com" when we get ip address of Facebook.com we are going to get the CIDR (Classless Inter-Domain Routing) which is a way of grouping or summarizing multiple IP addresses into a single, simplified range. Through which we can block all the facebook ip addresses. We get CIDR using the command "whois 157.240.22.35 | grep CIDR" we get the CIDR address(157.240.0.0/16), now we are taking sudo user access using the command "sudo -i" and after we are going to block outbound traffic to that server using the 'IPTables' which is a command-line interface used to configure and manage the Linux kernel firewall. Once we have CIDR of facebook we are blocking outbound traffic using the command "iptables -A OUTPUT -p tcp -d 157.240.0.0/16 -j DROP" This command is blocking all the outgoing traffic to the facebook.com from our network. After applying the IPTable command, we go to the Kali web browser and try to access the real facebook.com server but our browser cannot reach it, resulting in successfully blocking it.

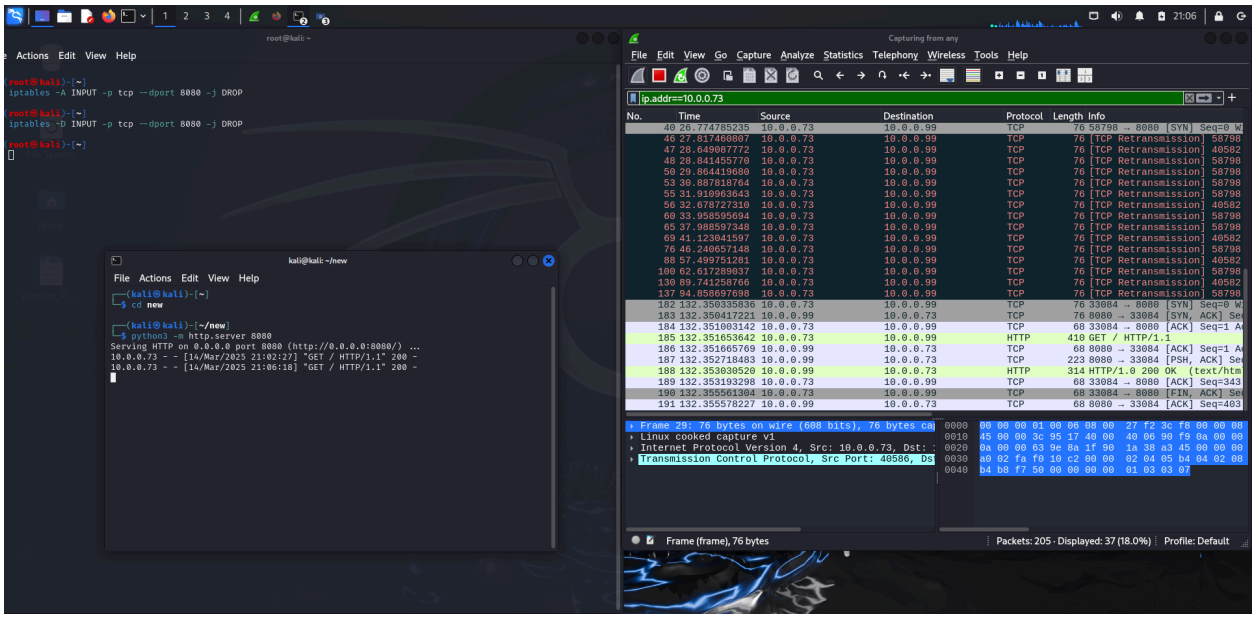


The above images show how to get Facebook's IP address and CIDR, and also the Outbound traffic stopped using the IPtables command. And the second image shows that we couldn't reach the real facebook server.

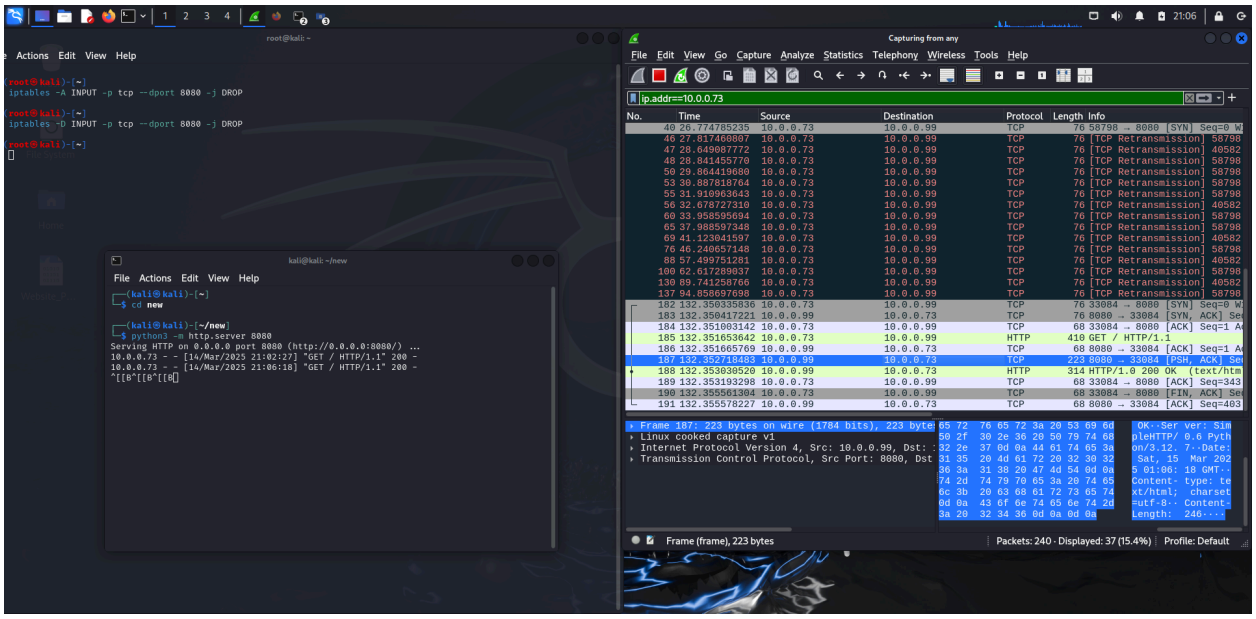
Now, we are going to block inbound traffic using IP tables in the Kali Machine. First we are going to open the bash terminal in Kali and type the command “cd new”. This command is used in the terminal to change the current working directory to a folder named new. Then we are going to host a website in Kali’s python server on port 8080 using the command “python3 -m http.server 8080” after hosting is started, after we are setting up wireshark and set to listen any traffic, and applying filter of the Ubuntu machines Ip address, because we are accessing the website using ubuntu browser. Now we are going to block the Inbound traffic on the port 8080 using the IPtables command “iptables -A INPUT -p tcp --dport 8080 -j DROP”. This will block all the incoming traffic to port 8080. After blocking the port, we tried to access the hosted web server from the ubuntu browser but we couldn't connect to it, due to a firewall blocking the port 8080. Now during the process we can see wireshark logs of the ubuntu ip-address as TCP retransmission which means the data packet never reached the receiver. Now we are going to unblock the port using the Ip tables command “iptables -D INPUT -p tcp --dport 8080 -j DROP” This iptables command removes (deletes) an existing firewall rule from the INPUT chain, specifically the rule that drops (blocks) TCP traffic arriving at port 8080. After deleting the firewall rule we again try to access the python hosted server using the Ubuntu web browser, Now Our Ubuntu host can connect to the fakeserver and also the wireshark no longer shows the TCP retransmission files instead it shows the TCP and HTTP communication between the machines because the firewall is no longer active and the connection is being established.



This Image above shows the hosted python server and the firewall is activated to block port 8080 traffic inbound traffic, and the wireshark is capturing the Lost TCP Retransmission Packets.

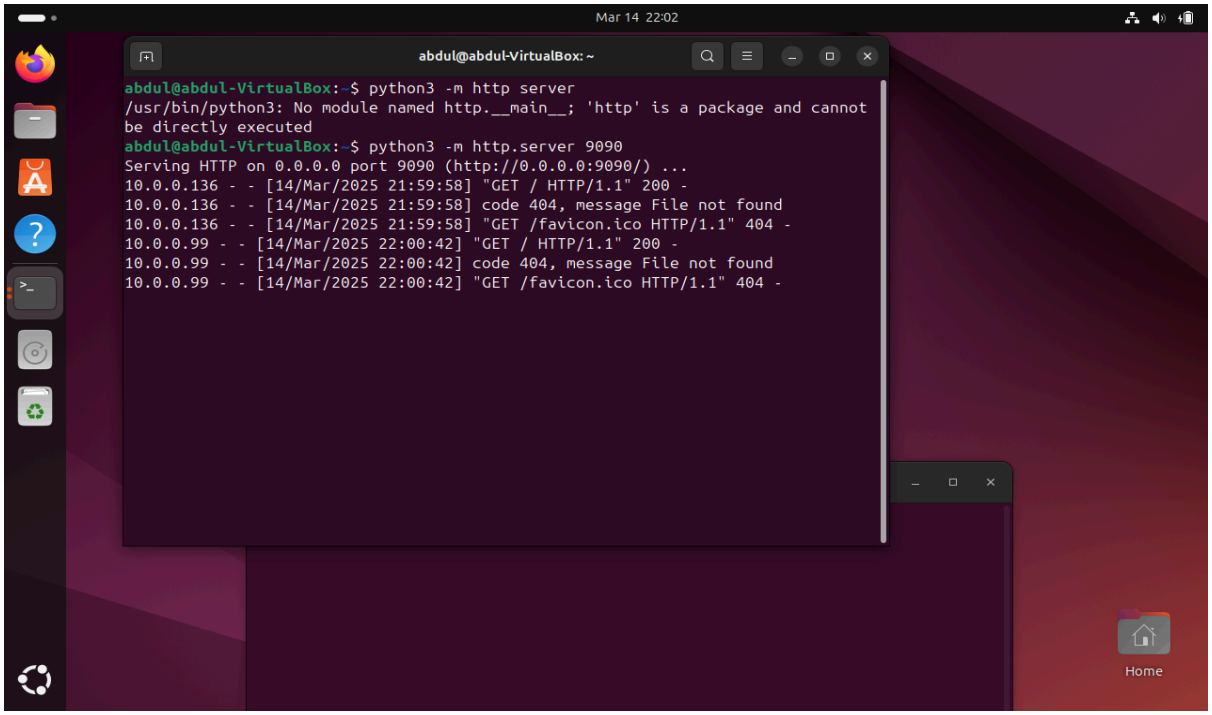


This image shows the Firewall rule is being removed using the IP tables command and Ubuntu host can connect to the fakeserver as soon as the port 8080 blockage has been removed. And also Wireshark showing the TCP and HTTP communication between the machines after removing the firewall rule.

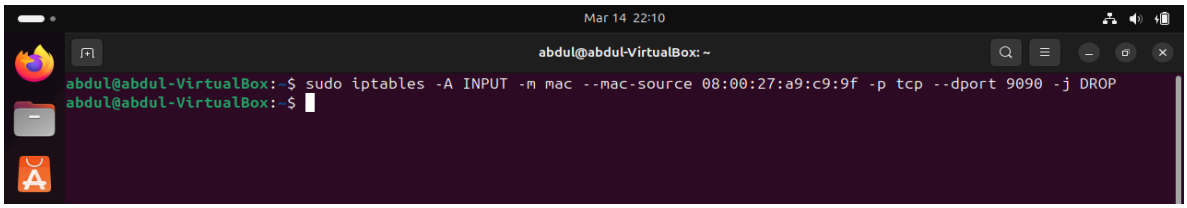


This image shows that the Ubuntu server is accessing our kali hosted Python server, and the highlighted blue area shows “SimpleHTTP/ 0.6 Python/3.12.”

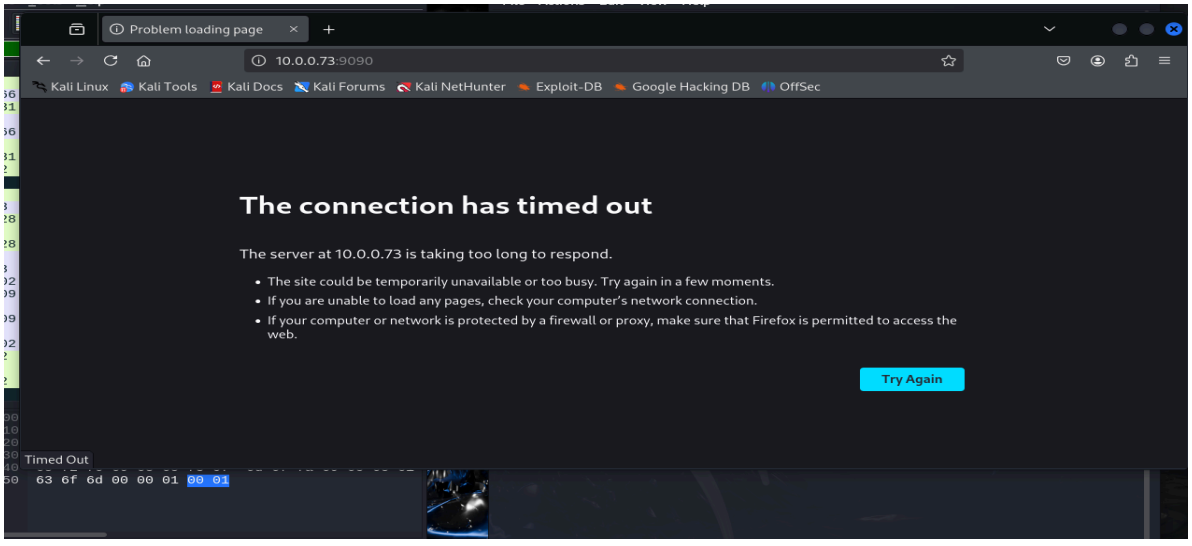
We have seen how to block outbound and inbound traffic using IP-Tables. Now, we are going to block mac address and see whether we can access the server even if our computers mac address is blocked. Now we are going to Ubuntu Machine and Open terminal and now we are hosting a python server using the command “python3 -m http.server 9090”. This will start hosting a python server over ubuntu ip address on port 9090 as shown in the image below.



Now, we are going to set up wireshark and start capturing the traffic on ‘any’ filter, and after that we are going to connect to the Ubuntu Python server using kali machine. Going on the kali browser and typing “http://10.0.0.73:9090”. And we could access the python server of Ubuntu from kali. Now we are going to block the mac address of kali on Ubuntu using the ip tables rule. So first we will know the mac of Kali using the “ip addr” command, which will show us the mac address of Kali which will be in link/ether, So the mac address of Kali is “08:00:27:a9:c9:9f”. After that we will go to Ubuntu terminal and type the command “sudo iptables -A INPUT -m mac --mac-source 08:00:27:a9:c9:9f -p tcp --dport 9090 -j DROP” This command will drop all the packets coming from the kali machine mac address on the port 9090. Now we try to access the Ubuntu hosted python server from kali. We would see that we couldn't access the hosted python server since our mac-address is blocked.

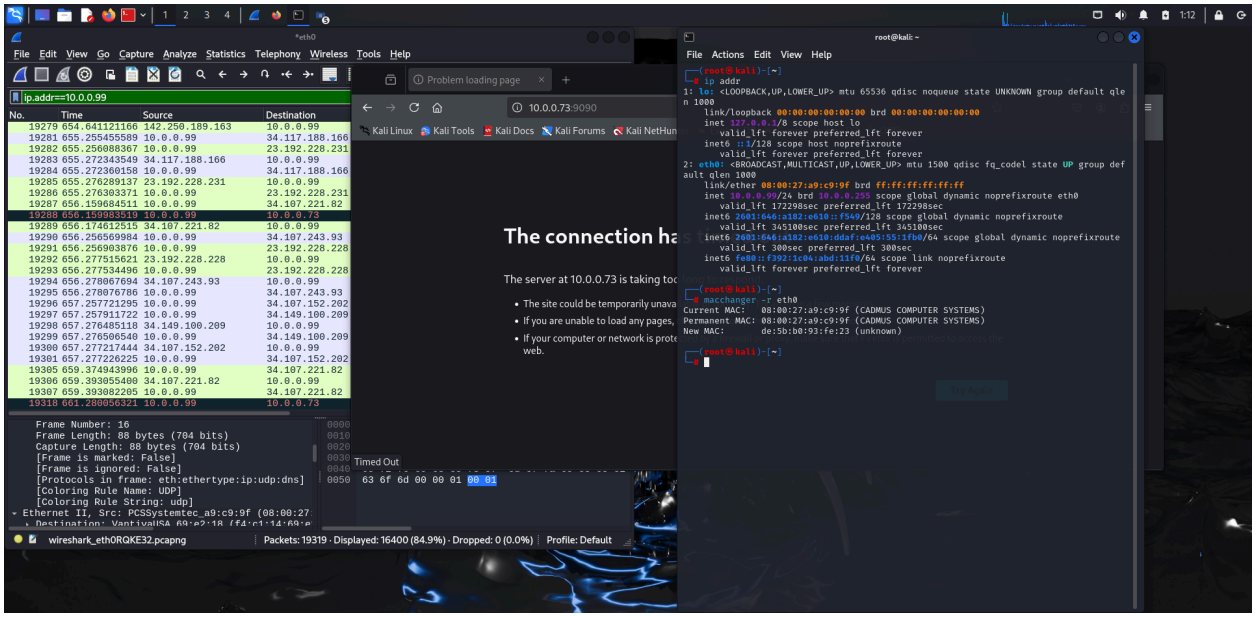


The first image shows that using iptables firewall rule we are blocking Kali machine mac address packets coming towards the port 9090, where the python server is hosted.



The second image shows that the server on ip 10.0.73 is not reachable on the kali machine since the mac address is blocked by the Ubuntu server.

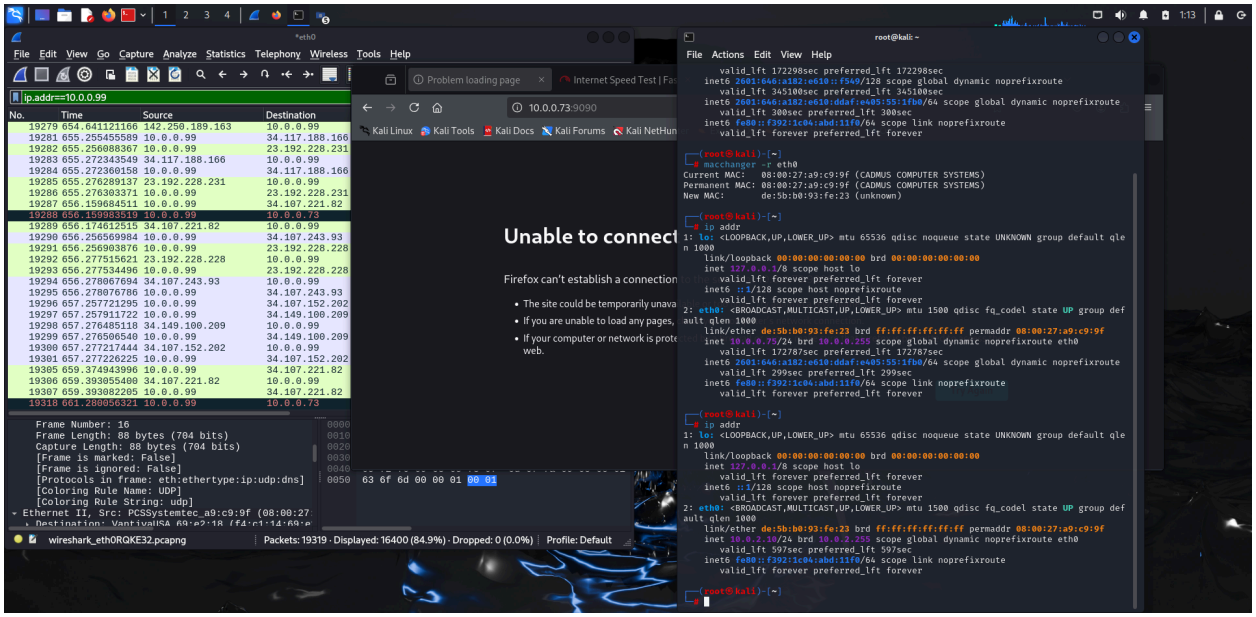
Now, since we are blocked by our mac address, we are going to change our mac address using Macchanger, it is a Linux utility used to view, modify, or spoof the MAC address of your network interface card (NIC) to bypass network restrictions. Now to change our mac address we type the command “macchanger -r eth0”, This will change our mac address and give us a new Mac which is “de:5b:b0:93:fe:23”.

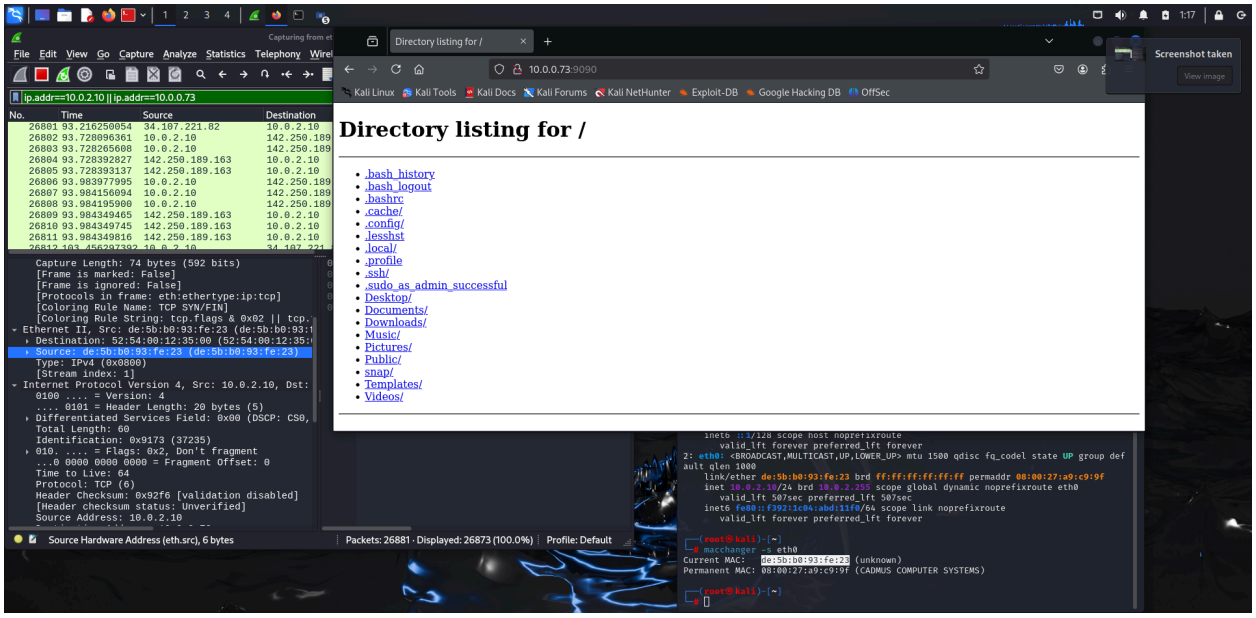


The above image shows that we have changed our mac address using the macchanger tool and obtained a new mac.

After we change our mac, we lose our connectivity on internet because our router or ISP usually assigns one ip to one mac address and our kali machine will have the old ip 10.0.0.99 linked to previous mac address, even after changing it to new mac address, so we have to reconfigure our kali network or restart the kali machine. After we reconfigure we are assigned with a new ip which is 10.0.2.10. And now we try to connect to the hosted python server from Ubuntu. We see that we can successfully access the server. So, using MAC addresses to filter traffic provides only a basic level of security and should not be relied upon as a primary defense. While it can prevent casual or unintentional access to a network, MAC addresses are easily spoofed using tools like macchanger. Attackers can observe allowed MAC addresses on the network and impersonate them to bypass the filter. Additionally, MAC filtering is ineffective across routers or the internet since MAC addresses operate only at the data link layer (Layer 2). For stronger security, combine MAC filtering with encryption, authentication, and firewall rules that inspect higher-layer traffic.

The image below shows us that the mac address has changed successfuly and we also have configured the ip- adress and checked using the ip addr command.





This image shows us that we can successfully access the hosted server from Ubuntu via kali machine after changing mac and ip address. This clearly tells that even if our old mac address is blocked, we can still spoof mac addresses and get access to the network. You can see that even the wireshark can see new mac address which is highlighted instead of the original permanent mac address.

After that we are going to use SSH (Secure Shell) is a cryptographic network protocol that allows secure communication between two computers over an insecure network like our internet. It's most commonly used to remotely log into another computer to execute commands on that system or transfer files securely or forward ports and tunnels securely. Now we are going to use ip tables to block inbound and outbound SSH network traffic. First we are going to start our SSH using the command “sudo service ssh start” where sudo gives access to root and asks for password before starting ssh service. Once started we are going to the Ubuntu terminal and type ”ssh kali@10.0.0.99”. This will build a ssh connection between both ubuntu and kali machines. When ssh is being started we could see the key exchange packets like Diffie hellman key exchange protocol being followed.

This image shows us the command “sudo service ssh start” is being used to activate the ssh service on the kali machine. And we are checking if ssh is active using the command “systemctl status ssh”. After the command we could see the green color active status being displayed.

Now, since we started our SSH now we are going to see what happens if we use ip tables to block the inbound traffic on the ssh connection on port 22. So we are going to use the command “sudo iptables -A INPUT -p tcp –dport 22 -j DROP”. This will block all incoming SSH connections by silently dropping packets on port 22. Any new SSH attempts to connect to the system will fail or timeout with no response. Since we are connected via SSH already, we may stay connected but it freezes or blocks established connections until the firewall rule is dropped.

The above image is showing me the firewall rule is being used to drop inbound SSH traffic suing the iptables command.

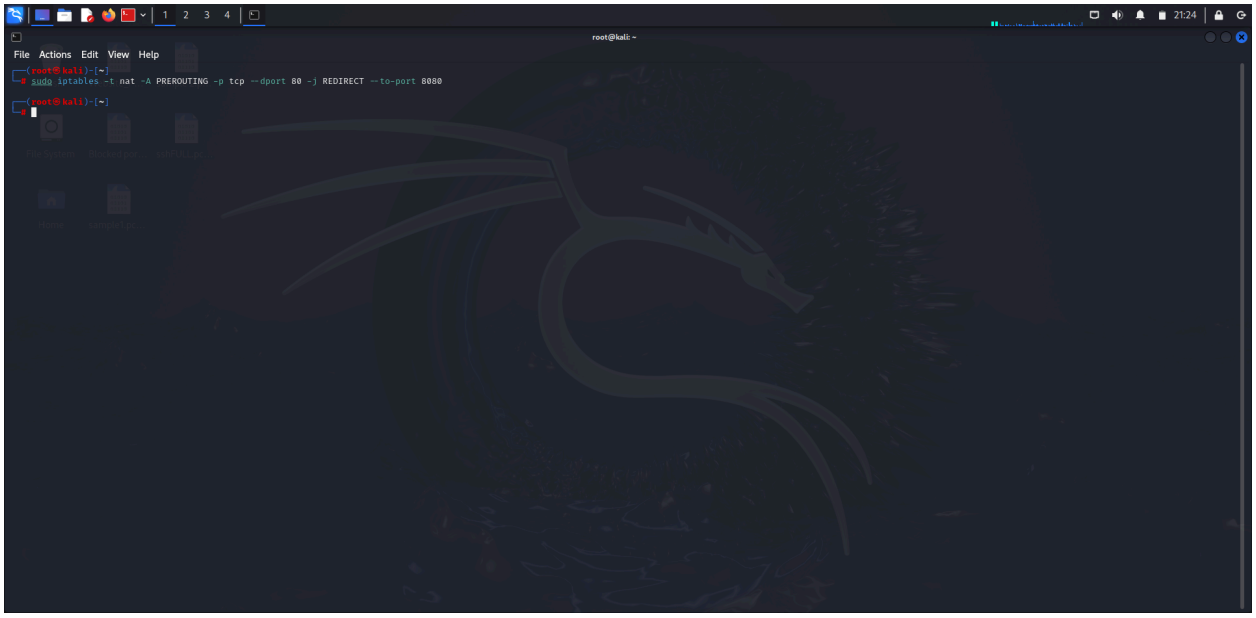
Once we remove the firewall rule that was blocking SSH port 22, the system will once again accept incoming SSH connection requests. This means new SSH sessions can be established normally, as long as the SSH server is running and configured correctly. Existing SSH sessions that were disrupted or previously blocked may also resume functioning, depending on whether the rule affected established connections. Essentially, dropping the rule restores network access on port 22 without interference from the firewall. This image shows the firewall rule is being removed.

Now, we are going to block outbound network traffic. We are using the command “sudo iptables -D OUTPUT -p tcp –sport 22 -j DROP”. This will block all output traffic from port 22. When you block outbound network traffic on SSH port 22, your machine will be unable to initiate new SSH connections to remote servers. This means that any attempt to connect via SSH will fail, and you'll typically see an error message like "Connection refused" or "Connection timed out." While any ongoing SSH sessions may continue as long as no new data transmission is needed, no new connections can be made to external systems on port 22. Essentially, blocking outbound traffic on port 22 restricts your ability to use SSH to remotely manage or transfer files to and from other machines, unless the block is removed. This ssh will stop working as it happened in the inbound traffic blockage. To remove the blockage we will be using the command “sudo iptables -D OUTPUT -p tcp –sport 22 -j DROP”.

This image shows the ip rule being applied to block outbound ssh traffic and also unlocking it.

Now,we are going to create a firewall rule to redirect any inbound request for port 80, and send this traffic to your python server on port 8080. We are going to sue the iptables rule “sudo iptables -t nat Prerouting -p tcp – dport 80 -j REDIRECT –to-port 8080” The command tells the firewall to intercept all incoming TCP traffic destined for port 80 (the default port for HTTP) and redirect it to port 8080 on the same machine. This is commonly used when a web application or proxy is listening on port 8080, but users or clients are trying to access it through the standard port 80. By applying this rule to the PREROUTING chain in the nat (Network Address Translation) table, the

redirection occurs before the routing decision is made, ensuring that the traffic is transparently forwarded without the client needing to know about the port change.



This image tells that the rerouting is done on kali machine from any inbound request for port 80, and send this traffic to the python server on port 8080.

Conclusion:

This assignment explores the use of Linux IPTables to manage network traffic for security purposes, simulating real-world attack and defense scenarios using Kali Linux (attacker) and Ubuntu (target) in VirtualBox VMs. It begins by hosting a fake Facebook website on Kali using Python's HTTP server and demonstrates how the Ubuntu system can be tricked into accessing it. The project then tests several firewall rules using IPTables, such as blocking outbound traffic to Facebook, blocking and unblocking inbound HTTP traffic on port 8080, and demonstrating TCP retransmissions using Wireshark. Further, the assignment shows MAC address filtering on Ubuntu and how MAC spoofing using Macchanger from Kali can bypass these filters, emphasizing the weakness of MAC-based security. Moreover, it explores blocking SSH connections, both inbound and outbound, using IPTables, showing how firewall rules affect secure shell connectivity. Throughout the process, network monitoring is carried out with Wireshark, and commands are executed through bash terminals, with a clear demonstration of firewall rule effects, spoofing risks, and traffic control capabilities. This can be turned into an attack by making a phishing site to get access to users login credentials and exploiting them for their own use. IPTables can be used as a powerful defensive tool to

block access to untrusted or malicious IP ranges, preventing data filtration or inbound attacks. Restrict services like SSH to specific IPs or MACs to reduce brute-force or unauthorized access attempts. Create logging rules to detect suspicious patterns or repeated access attempts. Redirect traffic for inspection, filtering, or honeypot deployment. Establish default-deny policies with specific allowlists to lock down all non-essential services. MAC address filtering is a network security technique that allows or denies network access to devices based on their MAC (Media Access Control) address, which is a unique hardware identifier assigned to a device's network interface card (NIC). However, this method is not secure because MAC addresses can be easily spoofed using tools like macchanger. Macchanger is a tool that allows you to change or spoof the MAC address of your network interface card (NIC). It can set the MAC address to a specific value or generate a random one. By changing their MAC address to one that is allowed, an attacker can bypass the filter, rendering the security measure ineffective. Therefore, while MAC address filtering can provide a basic level of control, it should not be relied upon as a sole security measure, as it can be easily circumvented. Firewalls mainly control access to networks by filtering traffic based on rules like IP addresses and ports. They don’t authenticate users or devices, meaning they don’t check who’s trying to connect. Firewalls don’t provide data encryption, so they don’t ensure data confidentiality, nor do they verify if data has been tampered with, which is needed for data integrity. They also don’t guarantee non-repudiation, meaning they don’t prevent someone from denying actions they’ve taken. While firewalls help block unwanted traffic providing us with the access control feature from x800 services, they work best when combined with other tools for full security, including encryption and authentication systems.